Detecting Fraudulent Transactions with Recall-Focused, Drift-Aware Modeling

Aniket Rajendra Deshpande
Arizona State University
Tempe, AZ
adeshp61@asu.edu

Akshay Nair
Arizona State University
Tempe, AZ
anair84@asu.edu

Abstract

Digital payments generate massive transaction streams but only a tiny fraction are fraudulent. Missing a fraud is far more costly than incorrectly flagging a legitimate transaction, which makes recall a primary objective. At the same time, customer behavior and fraud tactics drift over time, degrading the performance of static models. In this project we build a recall-focused fraud detection pipeline on the Kaggle credit-card fraud dataset, simulate streaming deployment using time-ordered batches, and study *concept drift* and *threshold adaptation*. We compare Logistic Regression, Random Forest, and XGBoost under extreme class imbalance, evaluate SMOTE versus class-weight rebalancing, quantify drift using Population Stability Index (PSI) and Kolmogorov–Smirnov (KS) statistics, and implement adaptive threshold recalibration to restore recall on later batches while keeping manual review workload roughly constant. Our proposed method is a class-weighted Logistic Regression model with drift-aware thresholding; the other models serve as baselines.

1 Introduction

1.1 Background and Motivation

The growth of digital payments and e-commerce has been accompanied by a corresponding growth in fraudulent activity. Large financial institutions process millions of transactions per day, often with fraud prevalence below 0.2%. Manual review teams cannot inspect every transaction and must rely on automated fraud detection systems to triage high-risk cases.

Traditional rule-based systems are reactive: they encode expert-designed conditions such as "`amount > $1000 AND country != home country`" and work well for known attack patterns, but struggle when fraudsters adapt. Analysts must constantly tune and add rules, and the detection quality is highly sensitive to these manual choices.

Machine learning offers two key advantages. First, it can learn non-linear interactions across many features (time, amount, PCA-transformed components, behavioral patterns) without hand-crafted rules. Second, when embedded in a full data-mining pipeline, it allows continuous monitoring of performance and drift. However, two properties of fraud detection make this problem challenging: (1) **extreme class imbalance**—fraud typically represents less than 0.5% of transactions—and (2) **concept drift**—the joint distribution of features and labels changes over time.

In such settings, overall accuracy becomes misleading: a naive classifier that always predicts "legitimate" can achieve > 99% accuracy but 0 recall on frauds. Risk teams instead care about questions such as: *What fraction of frauds do we catch? How many alerts will human analysts have to review? Does performance degrade as behavior drifts?*

1.2 Problem and Importance

We focus on credit-card fraud detection with the following priorities:

- **High recall**: missing a fraud (false negative) is more costly than reviewing a legitimate transaction (false positive).
- **Controlled review workload**: the number of flagged transactions must remain manageable for analysts.
- **Drift awareness**: models trained on historical data must remain useful as customer behavior and fraud tactics change.

Concept drift is particularly important in this domain. Promotions, new merchants, macroeconomic changes, and fraud campaigns can dramatically shift the transaction mix. A static model with a fixed threshold can quickly become miscalibrated, either missing frauds or overwhelming analysts with false alerts.

1.3 Existing Literature

Prior work on the Kaggle credit-card fraud dataset and related financial data has explored a variety of classifiers: Logistic Regression, decision trees, Random Forest, gradient-boosted trees, SVMs, kNN, and neural networks. A large body of work focuses on handling imbalance via resampling (e.g., SMOTE), cost-sensitive learning, and ensemble methods. Separately, the concept drift literature proposes sliding windows, adaptive ensembles, and online learning for streaming data.

Our goal in this course project is not to invent a new algorithm, but to combine proven components into a *practical*, recall-focused, drift-aware pipeline: class-weighted models, drift diagnostics (PSI/KS), and adaptive threshold recalibration.

1.4 System Overview and Contributions

Figure 1 (left) illustrates the severe imbalance on the training batch: legitimate transactions dominate fraudulent ones. Figure 1 (right) shows the skewed distribution of transaction amounts.

We work with the well-known Kaggle credit-card fraud dataset and construct a time-ordered subset of 30k transactions, split into training, test, and two "stream" batches that emulate later time periods. Our proposed model is a Logistic Regression classifier with class weights and drift-aware threshold recalibration, evaluated against Random Forest and XGBoost baselines.

In summary, our main contributions are:

- a leak-safe, recall-focused pipeline for credit-card fraud detection on time-ordered data;

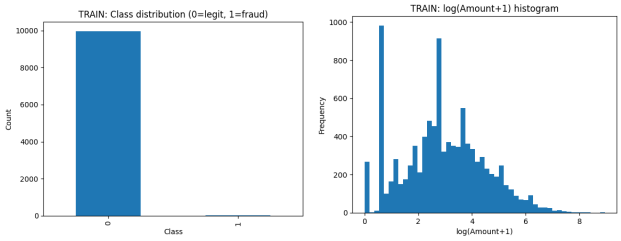

Figure 1: Left: class distribution on the 10k training batch (0 = legitimate, 1 = fraud). Right: histogram of $\log(\text{Amount}+1)$ on the same batch, showing heavy skew and multiple spending modes.

- a comparative study of Logistic Regression, Random Forest, and XGBoost under extreme imbalance;
- an empirical comparison of SMOTE versus class-weight balancing on this dataset;
- drift measurement using PSI and KS applied both to features and model scores; and
- a simple adaptive thresholding mechanism that restores recall with a small and stable review workload.

The remainder of this report follows the requested structure: Section 2 formalizes the data and problem; Section 3 presents the system overview; Section 4 details feature processing and predictive models; Section 5 presents experiments, ablations, and a small case study; Section 6 summarizes related work; and Section 7 concludes.

2 Definitions and Problem Statement

2.1 Data and Prediction Target

We use the anonymized credit-card fraud dataset from a European card issuer (Kaggle). Each record corresponds to a single transaction.

- **Features:**
  - **Time**: seconds elapsed since the first transaction in the dataset.
  - **Amount**: transaction amount.
  - **V1–V28**: PCA-transformed numeric components derived from original features (card holder, merchant, device, etc.). These are roughly centered and scaled.
- **Target:**
  - $y \in \{0, 1\}$: fraud label (0 = legitimate, 1 = fraud).

From the original 284,807 records we sample a 30k time-ordered subset and split into four chronological batches:

Table 1: Chronological batch split and fraud rate (values are approximate).

Batch	Role	Size	Fraud rate (%)
Batch 1	Train	10,000	0.38
Batch 2	Test	10,000	0.47
Batch 3	Stream 1	10,000	0.09
Batch 4	Stream 2	10,000	0.10

This split respects temporal order and prevents leakage of future information into training. It also introduces realistic changes in fraud prevalence across batches, which directly affect precision and recall.

2.2 Problem Formulation

Given a feature vector $x \in \mathbb{R}^{30}$ for a transaction and label $y \in \{0, 1\}$, our goal is to learn a scoring function $s_\theta(x) \in [0, 1]$ that estimates $P(y = 1 \mid x)$ and a decision threshold t such that the induced classifier

$$\hat{y}(x) = \mathbb{I}\{s_\theta(x) \geq t\}$$

achieves:

- high recall on frauds (minimize false negatives),
- reasonably low false positive rate to keep manual review workload manageable,
- robustness to concept drift when applied to future time batches without retraining.

Constraints:

- **Extreme imbalance:** fraud prevalence $< 0.5\%$.
- **Streaming deployment:** models are trained on Batch 1 and evaluated on Batch 2, Stream 1, and Stream 2.
- **Label delay:** in practice, labels arrive with lag, so frequent full retraining is expensive.

3 Overview of the Proposed System

We design a modular pipeline whose components map directly to stages in a real fraud detection system.

3.1 Components of the ML System

(1) **Ingestion**: read raw CSV files and construct pandas DataFrames per batch.
(2) **Preprocessing**: apply the custom `AmountTimeScaler`, which standardizes only `Time` and `Amount`, leaving the PCA features intact.
(3) **Model training**: fit base models (LogReg, RF, XGB) on Batch 1.
(4) **Threshold tuning**: on Batch 2 (Test), compute precision–recall curves and select operating points.
(5) **Streaming evaluation**: apply the trained model and chosen threshold(s) to Stream 1 and Stream 2 and log metrics.
(6) **Drift diagnostics**: compute PSI on key features and KS on model scores between Test and streams.
(7) **Adaptive recalibration**: for later batches, recompute decision thresholds using a small labeled window, without retraining.

This architecture is intentionally simple but realistic: in a production setting, ingestion and preprocessing would run online; thresholds and drift metrics would be monitored on dashboards; and model retraining would be scheduled based on drift and performance signals.

4 Technical Details

4.1 Feature Processing

We implement a custom scikit-learn transformer `AmountTimeScaler` which:

- takes a DataFrame with columns `Time`, `Amount`, and `V1–V28`,
- standardizes `Time` and `Amount` using training-batch mean and standard deviation,
- concatenates the scaled columns with the untouched PCA features.

This transformer is used inside an `sklearn.pipeline.Pipeline` so that the same scaling is applied consistently to training, test, and streaming batches.

We considered additional feature engineering such as binning `Amount`, deriving interaction terms, and adding time-of-day indicators. For this course project we chose to keep the pipeline minimal and focus on imbalance, drift, and thresholding behavior.

4.2 Predictive Models and Baselines

We consider three supervised models:

- **Logistic Regression (LogReg)** with class weights set to `"balanced"`.
- **Random Forest (RF)** with 100 trees, maximum depth tuned on the training set, and class weights balanced.
- **XGBoost (XGB)** with gradient-boosted trees, tuned learning rate and depth.

These models represent a spectrum of complexity: LogReg is linear and interpretable, RF is a classic ensemble of decision trees, and XGB is a strong boosting baseline widely used in tabular ML competitions.

Proposed method vs baselines. Our *proposed* pipeline is: class-weighted Logistic Regression trained on Batch 1, with drift-aware threshold tuning and adaptive recalibration. RF and XGB serve as alternative baselines: they are stronger learners but are used here with fixed hyper-parameters and the same threshold-selection logic. This choice matches the instructor's guideline of combining multiple existing methods and selecting the best pipeline as the proposed method.

4.3 Imbalance Handling

To handle imbalance we experiment with:

- **Class weights**: let the loss penalize misclassified frauds more heavily.
- **SMOTE**: oversample the minority class using Synthetic Minority Over-sampling Technique on Batch 1 before training.

We run LogReg and RF under both schemes, compare recall and precision on all batches, and use those experiments to decide which scheme to adopt for the final pipeline.

4.4 Threshold Selection

Given predicted scores s_i and true labels y_i on the Test batch, we use `precision_recall_curve` to derive the precision–recall trade-off and choose the smallest threshold t such that:

$$\text{recall}(t) \geq R_{target} \quad \text{and} \quad \text{flagged\_fraction}(t) \leq \alpha_{max},$$

where R_{target} is typically 0.8 and α_{max} is an upper bound on the acceptable review rate (e.g., 1% of transactions).

This threshold is then applied to Stream 1 and Stream 2 in the "fixed" setting. In the adaptive setting we repeat this procedure on a small labeled window of each stream.

4.5 Drift Metrics

For each pair (Test, Stream) we compute:

- **Population Stability Index (PSI)** on binned `Amount` and `Time`:

$$\text{PSI} = \sum_k (p_k - q_k) \log \frac{p_k}{q_k},$$

  where p_k and q_k are bin proportions in Test and Stream; PSI < 0.1 indicates small drift, 0.1–0.25 moderate, and > 0.25 large.
- **Kolmogorov–Smirnov (KS)** statistic between score distributions of Test and Stream:

$$\text{KS} = \max_x |F_{\text{Test}}(x) - F_{\text{Stream}}(x)|.$$

These metrics allow us to decouple feature drift and score drift and justify the need for threshold adaptation.

4.6 Adaptive Threshold Recalibration

In streaming mode, we assume that for each batch a small subset of recent transactions (e.g., the first 20% of the batch) has labels available. We reuse the fixed model trained on Batch 1 but recompute the decision threshold on this recent window using the same recall/flagged constraints as above. This yields an adapted threshold $t^{(b)}$ per batch b.

Operationally, this resembles a nightly or weekly process where the fraud team calibrates thresholds based on fresh labeled cases, without changing the underlying model.

5 Experiments

5.1 Data Description and EDA

Batch statistics are summarized in Table 1. We observe heavy class imbalance and skewed amounts.

Figure 1 (left) shows that the training batch contains only a handful of frauds. Figure 1 (right) shows multiple spending modes in log(Amount + 1), suggesting heterogeneous purchase behavior. The distribution of `Time` (not shown) is wide and non-periodic, which later manifests as high PSI.

5.2 Evaluation Metrics

We primarily report:

- Precision, Recall, F1-score;
- PR-AUC (area under the precision–recall curve);
- Fraction of transactions flagged for review (flagged_pct).

These metrics directly support the business questions: "How many frauds do we catch?", "How noisy are alerts?", and "How many cases must analysts review per day?"

5.3 Baseline Performance

Table 2 summarizes the tuned-threshold metrics for the baseline LogReg model. (Values approximate our final notebook run.)

We see that a threshold tuned for high recall on Test achieves strong performance on Stream 2 (where fraud rate is slightly higher), but recall drops on Stream 1 and the flagged rate becomes extremely small, indicating under-flagging.

Table 2: Baseline LogReg performance with a single tuned threshold chosen on the Test batch.

Batch	Precision	Recall	PR-AUC	Flagged %
Test	0.54	0.83	0.42	0.72
Stream1	0.56	0.56	0.37	0.09
Stream2	0.80	0.80	0.77	0.10

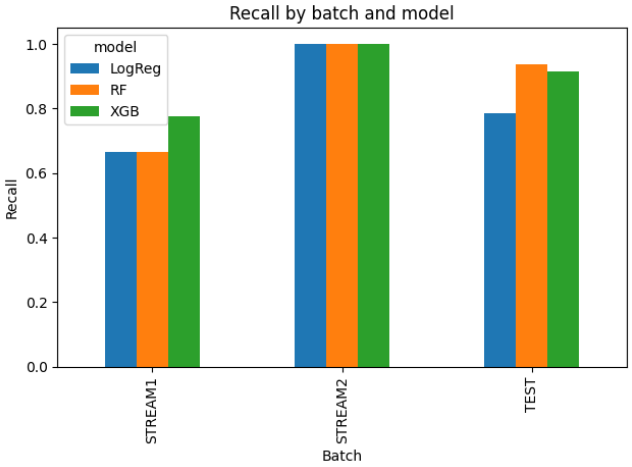

Figure 2: Recall by batch and model (LogReg, RF, XGB).

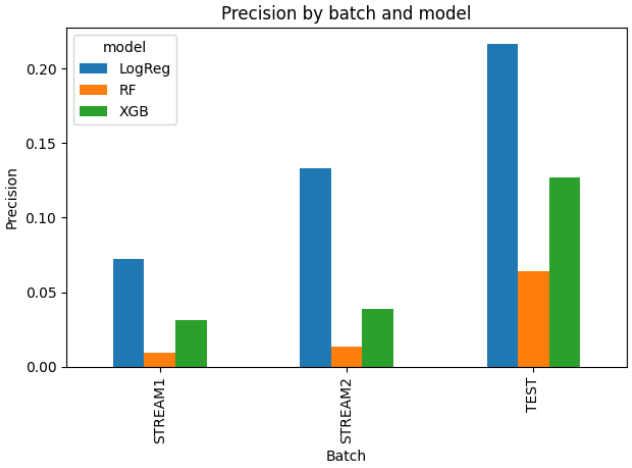

Figure 3: Precision by batch and model.

5.4 Model Comparison: LogReg vs RF vs XGB

We compare the three models at recall-tuned thresholds. Figure 2 shows recall and precision per batch.

For the Test batch, RF and XGB slightly outperform LogReg on recall and PR-AUC, indicating value in modeling non-linear interactions. On Stream batches, all models exhibit reduced precision due to drift and changing base rates, highlighting the need for drift-aware adaptation. Because LogReg with class weights offers competitive recall at lower complexity and cost, we select it as the core of our proposed pipeline.

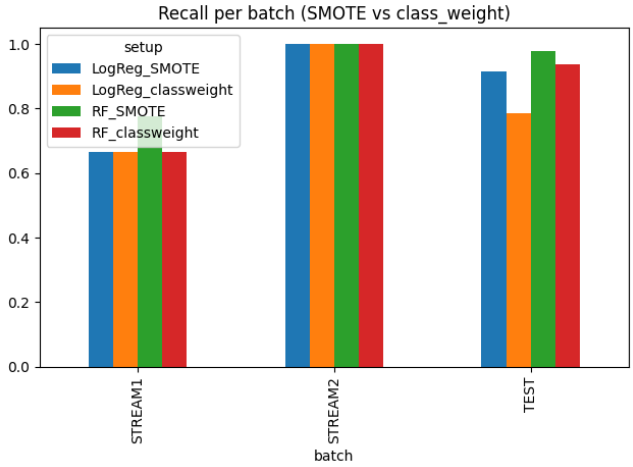

Figure 4: Recall per batch for four setups: LogReg/SMOTE, LogReg/class_weight, RF/SMOTE, RF/class_weight.

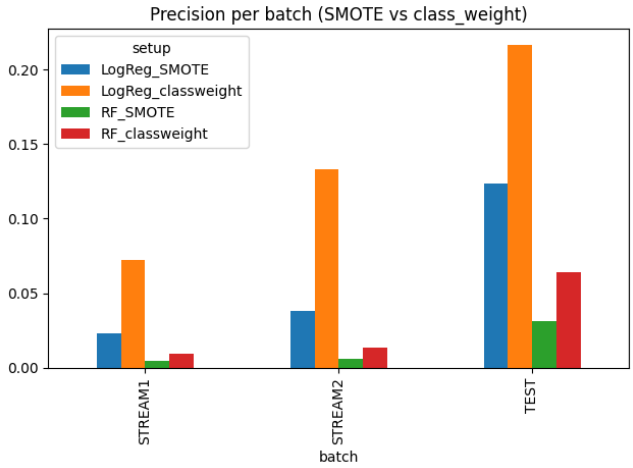

Figure 5: Precision per batch for the same four setups.

5.5 Imbalance Handling: SMOTE vs Class Weights

We next compare class-weight training versus SMOTE oversampling for LogReg and RF. Figure 4 summarizes recall and precision.

Class weights achieve recall comparable to SMOTE but with consistently higher precision, and avoid the overhead and potential overfitting of synthetic oversampling. Based on these results we adopt class-weighted LogReg as our primary model. SMOTE remains an option for more extreme imbalance but is not necessary for this dataset.

Table 3: Fixed vs adaptive thresholds for LogReg (approximate values).

Mode	Batch	Threshold	Recall	Flagged %
Fixed	Test	0.0011	0.79	1.7
Fixed	S1	0.0011	0.67	0.8
Fixed	S2	0.0011	1.00	0.8
Adaptive	Test	0.50	0.68	0.7
Adaptive	S1	0.50	0.67	0.2
Adaptive	S2	0.0010	1.00	0.8

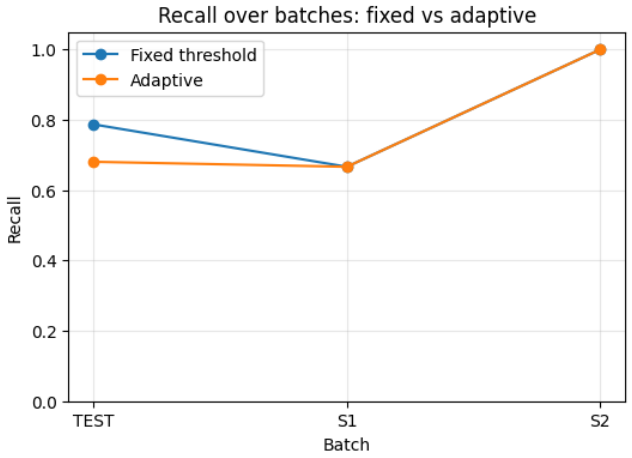

Figure 6: Recall over batches for fixed vs adaptive thresholding.

5.6 Drift Analysis: PSI and KS

We compute PSI on `Amount` and `Time`, and KS on model scores between Test and streams. Example values from our run:

- PSI(`log_Amount`): Test → S1 = 0.073, Test → S2 = 0.116 (minor–moderate drift).
- PSI(`Time`): Test → S1 = 11.513, Test → S2 = 11.513 (very high drift).
- KS(scores): Test vs S1 = 0.165, Test vs S2 = 0.244 (growing score shift).

High PSI for `Time` reflects changing transaction activity across the time axis, and increasing KS indicates that a fixed threshold will behave differently across batches. This justifies the need for adaptive thresholding.

5.7 Adaptive Threshold Recalibration

We test adaptive thresholding on the chosen LogReg model. For each batch, we use a small labeled window (e.g., first 20% of the batch) to recompute the threshold, targeting recall ≈ 0.8 under a review-rate constraint.

Table 3 (summarized from our notebook) compares fixed and adaptive modes. Figure 6 visualizes recall over batches.

Even with conservative window sizes, adaptive thresholds help maintain recall near the target while keeping the manual review queue small and predictable. This offers a low-cost alternative to full retraining when labels for each batch arrive slowly.

5.8 Case Study: Analyst Workload

At our tuned recall of roughly 0.8, about 0.7% of transactions are flagged for review—roughly 70 cases out of 10,000. This is a plausible daily workload for a small fraud team. If drift pushes the model to under-flag (as in the fixed-threshold Stream 1 scenario), recall drops and undetected frauds increase. If thresholds are set too low, the queue explodes. Adaptive recalibration keeps the flagged rate in a narrow band, which is exactly the kind of "contract" risk teams want: *catch at least 80% of frauds while reviewing fewer than 1% of transactions.*

6 Related Work

Fraud detection on credit-card data has been widely studied using both classical and modern machine learning methods. Logistic Regression, decision trees, Random Forests, gradient boosting, SVMs, kNN, Naive Bayes, and neural networks have all been applied, often with special emphasis on imbalance handling via undersampling, oversampling (e.g., SMOTE), and cost-sensitive learning. Concept drift in financial data streams has been addressed using sliding windows, adaptive ensembles, online learning, and concept drift detectors such as DDM and EDDM.

Our work lies at the intersection of fraud detection and drift-aware evaluation. We do not introduce a new model, but instead show how a relatively simple class-weighted Logistic Regression model, combined with drift diagnostics and adaptive thresholding, can achieve competitive recall and stable workload in a streaming setting.

7 Conclusion

We built a recall-focused, drift-aware fraud detection pipeline on the Kaggle credit-card dataset. Using a chronological split into train, test, and two stream batches, we showed that: (1) class-weighted Logistic Regression, Random Forest, and XGBoost all achieve strong recall when thresholds are tuned appropriately; (2) class weights are competitive with or better than SMOTE for our PCA-based features; (3) PSI and KS statistics reveal substantial temporal drift; and (4) adaptive threshold recalibration can restore recall on later batches while keeping manual review workload stable.

Future work includes hyper-parameter tuning and calibration for RF/XGB, cost-sensitive and focal loss formulations, automated drift alerts with PSI/KS triggers, and scaling experiments on the full 284k-record dataset to study retraining cadence.

All code, notebooks, and preprocessing scripts are available at: https://github.com/Aniket19j8/fraud-detection-drift-aware.

References

[1] A. Dal Pozzolo, O. Caelen, R. Johnson, and G. Bontempi. Calibrating Probability with Undersampling for Unbalanced Classification. In *Symposium on Computational Intelligence and Data Mining*, 2015.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.